

# USS SpiderBot: Robot Cuadrúpedo Articulado

Solemne 3 — Taller de Programación I

## **Integrantes:**

Moisés Amundarain, Camila Aravena, Fernando Ramírez,  
Juan Pablo Vargas, María José Santander

Ingeniería Civil Informática  
Facultad de Ingeniería  
**Universidad San Sebastián**

23 de Junio, 2026



UNIVERSIDAD  
**SAN SEBASTIAN**  
Ilumina el futuro

# Introducción y Propósito

## Definición de Proyecto

El **USS SpiderBot** es un robot caminador cuadrúpedo de **8 grados de libertad (8-DoF)** orientado al reconocimiento autónomo en zonas de catástrofe y escombros.

## Desafío Tecnológico

Sustituir ruedas por extremidades articuladas para sortear desniveles físicos de forma estable utilizando control de bucle cerrado.



## Chasis Doble Deck

- **Placa Inferior ( $r = 65\text{mm}$ ):** Aloja perimetralmente los brackets de cadera.
- **Placa Superior ( $r = 65\text{mm}$ ):** Cuenta con cuna central para protoboard de 400 puntos y marcos protectores.

## Configuración Cinemática

Juntas en paralelo horizontal (**Pitch-Pitch**) para maximizar torque vertical y evitar cizalle dinámico.

## Tolerancias de Precisión (FDM)

- **Cavidad Servo:**  $+0.3\text{mm}$  de holgura nominal.
- **Bolsillos de Doble Profundidad:** Dual-depth (30mm abajo para cables, 25mm arriba).
- **Single-Arm Horn Locks:** Ranuras chavetadas que encastran el horn estriado a presión.

## Bus I2C Compartido

El microcontrolador **ESP32 DevKit V1** comanda la lógica I2C en paralelo hacia dos esclavos:

- ① **PCA9685 (0x40)**: Driver PWM de 12-bit.
- ② **MPU6050 (0x68)**: IMU de 6 ejes.

## Aislamiento de Potencia (UBEC)

- Los 8 servos consumen picos de hasta 4A combinados.
- **UBEC (5V/3A)**: Regula y separa la línea de fuerza externa de la bornera del PCA9685.
- Previene caídas de tensión y reinicios (\*brownouts\*) del procesador ESP32.
- **GND Unificado**: Referencia de voltaje común.

## Planificación Cooperativa

Sustitución de llamadas bloqueantes (`time.sleep_ms()`) por corrutinas no bloqueantes concurrentes:

- `sensor_updater()`: Adquisición MPU6050 y Sonar a 20 Hz.
- `locomotion_loop()`: Lazo de cinemática y control servo.
- `start_server_task()`: Socket TCP HTTP del dashboard.

## Estado Compartido y Seguridad

- **state.py**: Singleton global que comparte la telemetría y órdenes de control en tiempo real.
- **Lazo de Seguridad**: Si el sonar detecta un obstáculo a menos de 15 cm, se cancela la locomoción en menos de 25 ms, regresando a pose segura con pitido acústico.

# Algoritmo de Locomoción: Marcha de Gateo (Crawl Gait)

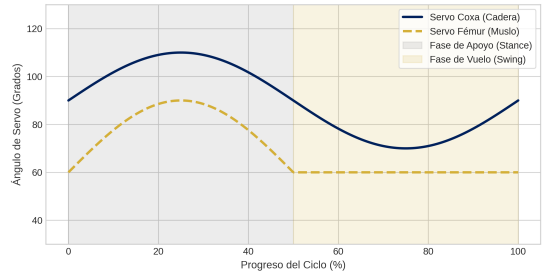
## Polígono de Sustentación

Estabilidad estática garantizada: en fase de oscilación (swing), el centro de masa (CoM) siempre reside dentro del triángulo formado por los tres puntos de apoyo en el suelo.

## Ciclo Cinemático (5 Fases)

Secuencia repetitiva: Stance Shift general → Paso Pata FR → Paso Pata RR → Paso Pata FL → Paso Pata RL.

Perfiles Angulares del Ciclo de Marcha (Crawl Gait)



*Perfiles de articulación simulados en Python*

# Lazo Cerrado de Estabilización Inercial Activa

## Compensación Proporcional

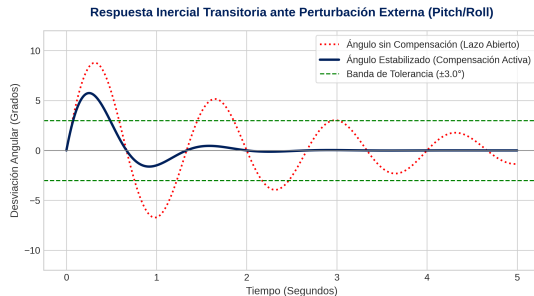
Inyección de offsets angulares dinámicos en los servos de rodilla (fémures) de las patas en contacto:

$$\theta_{compensado} = \theta_{base} \pm (K_p \cdot \theta_{sensor})$$

Donde el coeficiente  $K_p = 1.2$  compensa proporcionalmente la caída del chasis medida por la IMU a 20 Hz.

## Tolerancia Activa

La compensación se activa si la inclinación medida supera la banda muerta de  $\pm 3^\circ$ .



*Simulación del comportamiento transitorio inercial*

# Dashboard Web Embebido de Telemetría y Mando

## Dashboard Autocontenido (Glassmorphism)

- Archivo HTML, CSS y JS unificado de 24 KB servido de forma asíncrona.
- **100 % Offline:** Sin CDNs o llamadas de red externas para evitar dependencias locales.
- Enrutamiento por bloques de 512 bytes para evitar desbordamientos en la memoria de la ESP32.

## Funciones de la Interfaz

- **Mando por Teclado:** Soporte de teclas físicas (W-A-S-D), barra espaciadora (parada) y tecla R (reposo).
- **Graficador SVG:** Visualizador vectorial 2D en tiempo real del plano inclinado del chasis inercial.
- **Telemetría en Vivo:** Lectura constante de distancia e inclinación.



## Simulador Virtual Wokwi

- Cableado digital completo de 8 servos y sensores en `diagram.json`.
- **Control Fallback Directo:** En ausencia del expansor PCA9685, el firmware conmuta a modulación PWM nativa de la ESP32.
- Conexión a través del canal virtual 'Wokwi-GUEST' para interactuar con el servidor real.

## Especificaciones de Slicing (Creality Print)

- **Altura de Capa:** 0.2 mm con PLA/PETG.
- **Líneas de Pared:** Mínimo 4 perímetros (esencial para la fuerza de los agujeros roscados M2).
- **Relleno:** 30 % en patrón **Gyroid** (giroidal) por su rigidez mecánica tridimensional isotrópica.
- Proyecto de plato listo guardado en `[spiderbot.3mf]`.

# Conclusiones y Trabajo Futuro

## Logros Obtenidos

- Integración exitosa de modelado CAD, electrónica embebida, control asíncrono y desarrollo de red en la ESP32.
- Estabilización proporcional inercial y evasión de obstáculos dinámicas completamente funcionales.
- Estructura paramétrica en OpenSCAD lista para laminación FDM.

## Mejoras Propuestas (Trabajo Futuro)

- **Cinemática Inversa (IK):** Control completo de 3-DoF por pata para trayectorias de pie exactas.
- **Visión Inteligente:** Integrar cámara web ESP32-CAM y algoritmos de detección local para rescates.
- **Marcha Dinámica:** Programar trote alternado con fases de vuelo.



UNIVERSIDAD  
SAN SEBASTIAN

---

Ilumina el futuro

**¡Muchas Gracias!**

Proyecto USS SpiderBot — Solemne 3